Algoritma & Struktur Data

Week 1 - Pointer & Array
Universitas Multimedia Nusantara

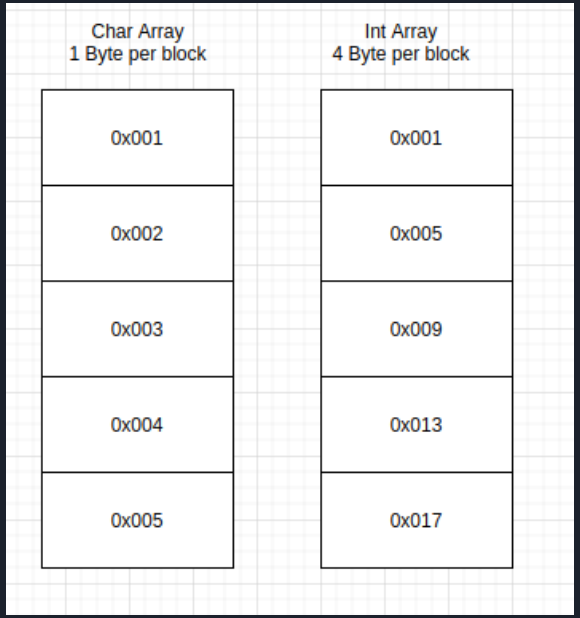
Outline

- Array
  - What
  - How
  - Why
- Pointer
  - What
  - Why
  - How
- Connection between Pointer & Array
- Guess the output :D

Array (What)

- Array is a set of homogeneous data which is stored in an contiguous memory sequence.

- Constraint (in c / c++) :
  - Must be homogeneous (meaning same data type with same size).
  - It use an integer as it's index to determine which address should be accessed.
  - ANY data type, struct, class can be used as an array except void.
  - Zero Based (it start with 0 as it's index)



Array (Why)

- Let's say you want to evaluate 1000 student score. Which one of the following is better
  - Creating 1000 Variables?
    - int student_score_1
    - int student_score_2
    - .....
    - int student_score_1000
  - Creating an array with 1000 index?
    - int student_score[1000]
- Let's say you want to access it, which one is easier?
  - Remember every 1000 variable you have?
    - student_score_1 = (something)
  - Access it with simple integer value
    - student_score[1] = (something)

Array (How)

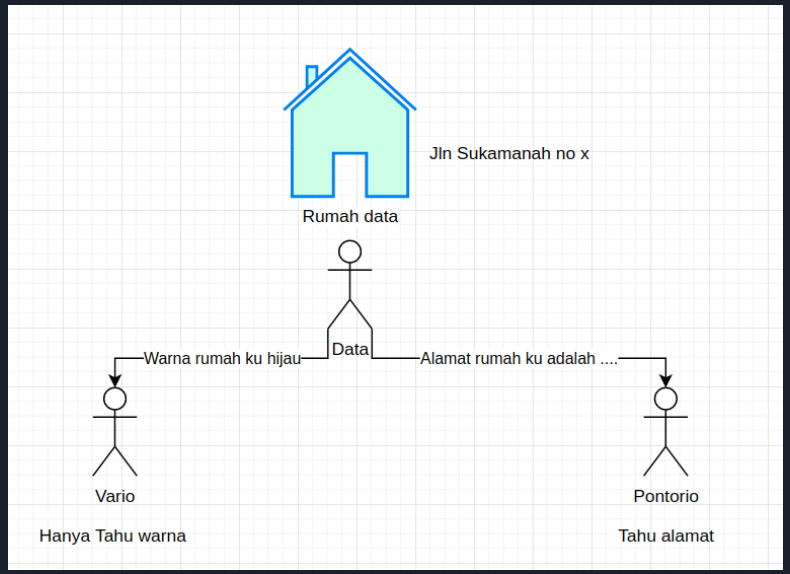
- Declaration format:
  - data_type name[length]
  - Ex: int student_score[1000]
- Assignment Operation
  - name[index] where index = integer
  - Ex input: scanf(“%d”,&student_score[0]);
- Accessing Operation
  - Ex Output: scanf(“%d”,student_score[0]);

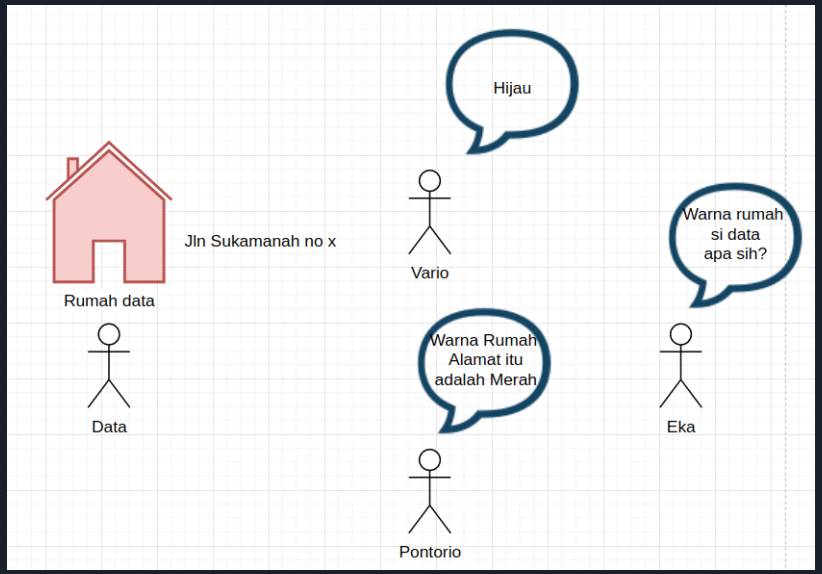
```
#include <iostream>
using namespace std;


int main() {
    int data[25];
    int i;
    for(i = 0; i < 20;i++){
        data[i] = i;
    }


    for(i = 0; i < 20;i++){
        printf("%d ",data[i]);
    }
    return 0;
}
```

Pointer (What & Why)

- Pointer as it name suggest, is a variable which store the address which they will point out.
- Constraint (in c / c++) :
  - It store address, not value.
  - Based on it's address, we can get the value of that address
  - If the value of pointed address is changed, it will also affect the c
  - It can only point to the variable with a same data type.
- Why and where is it used?
  - Real case study: we want to pass a variable to another function and modify that variable.
  - Basically, you only use pointer when you want to keep track a value at certain memory block.

Very oversimplified analogy





Pointer (How)

- Declaration format:
  - data_type *name
  - Ex: int *data
- Assignment Operation
  - data = &var (& Represent the address)
- Accessing pointer value
  - printf(“%d”,a); ← return address
- Accessing
  - printf(“%d”,*a); ← return value of the address

```
#include <iostream>
using namespace std;

int main() {
    int *a;
    int b = 10;
    a = &b;
    printf("%d %d\n",&b,b);
    printf("%d %d\n",a, *a);
    return 0;
}
```

```
#include <iostream>
using namespace std;


void square(int *data){
    *data = (*data) * (*data);
}

int main() {
    int a = 2;
    square(&a);
    printf("%d\n",a);
    square(&a);
    printf("%d\n",a);

    return 0;
}
```

Connection between Pointer & Array

- Pointer basically store the address.
- Pay attention to this statement, it store the address of array at index 5
  - pointer = &arr[5]
- Since we know that array has a contiguous address.
  - pointer[-1] ⇔ arr[4] ← meaning we access 1 address before initial address.
  - pointer[0] ⇔ arr[5] ← we access the initial referenced address
  - pointer[1] ⇔ arr[6] ← we access 1 address after initial referenced address
- The implementation?
  - Transforming the dimension of the array.
    - 1D array to 2D
    - 2D to 1D array

```
#include <iostream>
using namespace std;


int main() {
    int *pointer;
    int data[100];
    int i;
    for(i = 0; i < 20;i++){
        data[i] = i;
        printf("idx %d value= %d, idx %d address = %d\n",i,data[i],i,&data[i]);
    }

    pointer = &data[19];

    for(i = 0; i > -20; i--){
        printf("idx %d value= %d, idx %d address = %d\n",i,pointer[i],i,&pointer[i]);
    }
    return 0;
}
```

Array dimension

- Array dimension defined on how do we initialize the array, see these below example:
  - 1D array → int data[10], it store linear form of data (X).
    - Stack
    - Queue
    - Data storage
    - List of data
    - Tree
  - 2D array → int data[10][10]. It store data in a matrix form (X,Y).
    - Matrix
    - Table
    - X,Y cartesian Map.
    - Graph
  - 3D array → int data[10][10][10]. It store data in a far more complex form (X,Y,Z).
    - Position Unit (X,Y,Z)
  - 4D array? Care to show an example?
- One reminder, simple array initialization such as data[10], data[10][10], data[10][10][10] will always have continuous address.
- Array with malloc would be a different case though. (more on that later)

How to use multidimensional array?

```cpp
#include <iostream>
using namespace std;


int main() {
    int *pointer;
    int data[10][10];
    int i, j;
    for( i = 0; i < 5;i++){
        for( j = 0; j < 5;j++)
            data[i][j] = i*5+j;
    }
    for( i = 0; i < 5;i++){
        for( j = 0; j < 5;j++)
            printf("%d ",data[i][j]);
        printf("\n");
    }
    return 0;
}
```

```cpp
#include <iostream>
using namespace std;


void add_10(int data[10][10]){
    int i,j;
    for(i = 0; i < 5;i++){
        for(j =0; j<5; j++){
            data[i][j]+=10;
        }
    }
}

int main() {
    int data[10][10];
    int i, j;
    for( i = 0; i < 5;i++){
        for( j = 0; j < 5;j++)
            data[i][j] = i*5+j;
    }
    add_10(data);
    for( i = 0; i < 5;i++){
        for( j = 0; j < 5;j++)
            printf("%d ",data[i][j]);
        printf("\n");
    }
    return 0;
}
```

```cpp
#include <iostream>
using namespace std;


void add_10(int data[][10]){
    int i,j;
    for(i = 0; i < 5;i++){
        for(j =0; j<5; j++){
            data[i][j]+=10;
        }
    }
}

int main() {
    int data[10][10];
    int i, j;
    for( i = 0; i < 5;i++){
        for( j = 0; j < 5;j++)
            data[i][j] = i*5+j;
    }
    add_10(data);
    for( i = 0; i < 5;i++){
        for( j = 0; j < 5;j++)
            printf("%d ",data[i][j]);
        printf("\n");
    }
    return 0;
}
```

How to convert 1D to 2D array and revert it back?

- Remember how the address of the array is continuous?
- Meaning as long as we provide the first address, we can easily treat it as a 1D array.
- Given you remember the max size of the array

```
#include <iostream>
using namespace std;


void add_10(int *data){
    int i,j;
    for(i = 0; i < 25;i++){
        data[i]+=10;s
    }
}

int main() {
    int data[10][10];
    int i, j;
    for( i = 0; i < 5;i++){
        for( j = 0; j < 5;j++)
            data[i][j] = i*5+j;
    }
    add_10(&data[0][0]);
    for( i = 0; i < 5;i++){
        for( j = 0; j < 5;j++)
            printf("%d ",data[i][j]);
        printf("\n");
    }
    return 0;
}
```

Guess the output :D

THE END